



# Python Loops

#PythonNotes

# Loop Statements



# Loops in Python

- The loops in Python are used to execute a block of code or a part of the program several times.
- In other words, it execute a block of code many times.

# Why use loops?

- Suppose that you have to print table of 2, then you need to write 10 lines of code.
- By using the loop statement, you can do it by 2 or 3 lines of code only.

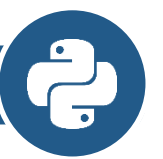
- Advantage of loops in Python
  - It saves code.
  - Faster execution of code.

# Loops

- While Loop Statements
- For Loop Statements
- Nested Loops Statements

# Points to Remember

1. In Loop we need Variable with Initial Value
  - `a = 1`
2. Loop Needs Condition
  - `a < 10`
3. Print Statement
  - `print(a)`
4. Loop required Increment or Decrement Value
  - `a = a + 1`
  - `a = a - 1`



# While Loop

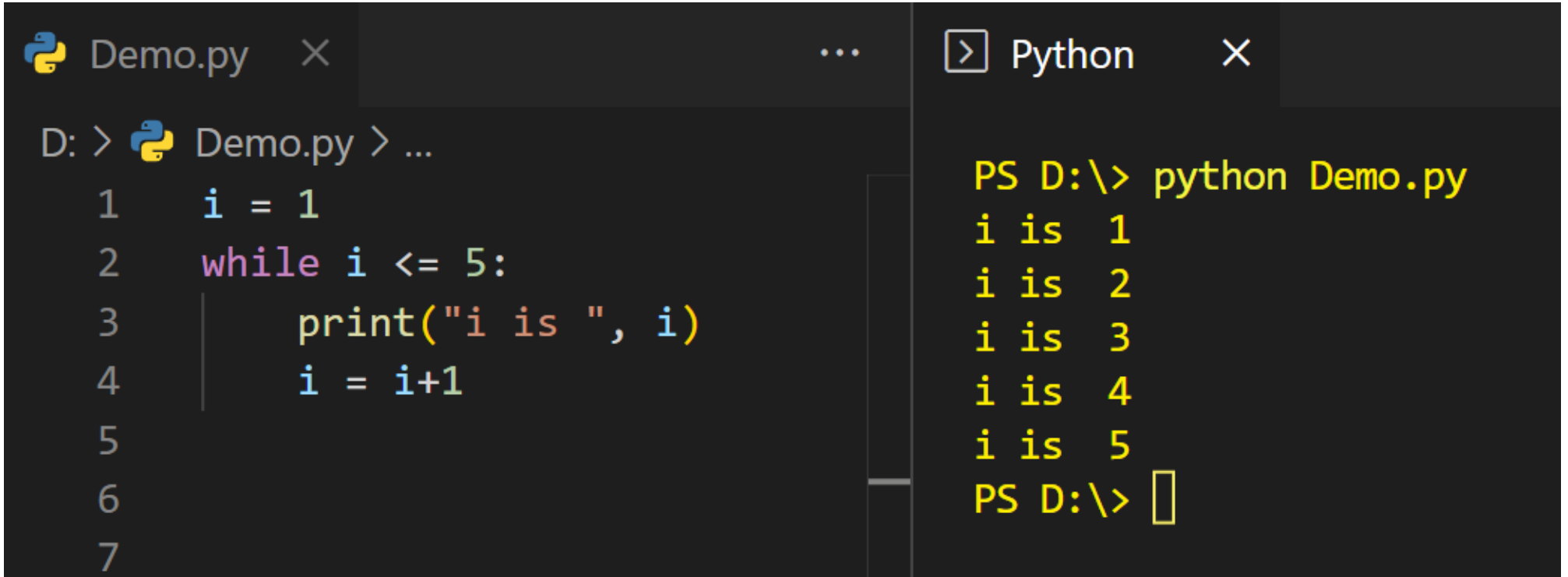


# while Loop Statements

- The while loop in Python is used to execute the part of program or statements many times.
- **Syntax:-**
  - variable initialization
  - while expression:
  - statement(s)
  - variable increment or decrement

# Example

```
i = 1
while i <= 5:
    print("i is ", i)
    i = i+1
```



The screenshot shows a code editor window titled 'Demo.py' and a terminal window titled 'Python'. The code editor contains the following Python code:

```
D: > Demo.py > ...
1  i = 1
2  while i <= 5:
3      print("i is ", i)
4      i = i+1
5
6
7
```

The terminal window shows the output of running the code:

```
PS D:\> python Demo.py
i is 1
i is 2
i is 3
i is 4
i is 5
PS D:\>
```

# Increment with 1 or 2 in Value

```
Demo.py x [play] [stop] [refresh] [menu]
D: > Demo.py > ...
1 i = 1
2 while i <= 5:
3     print("Hello World")
4     i = i+1
5
6

PS D:\> python Demo.py
Hello World
Hello World
Hello World
Hello World
Hello World
PS D:\> 
```

```
i = 1
while i <= 5:
    print("Hello World")
    i = i+1
```

```
i = 1
while i <= 5:
    print("i is ", i)
    # i=i+2
    i += 2
```

```
Demo.py x [play] [stop] [refresh] [menu]
D: > Demo.py > ...
1 i = 1
2 while i <= 5:
3     print("i is ", i)
4     # i=i+2
5     i += 2

PS D:\> python Demo.py
i is 1
i is 3
i is 5
PS D:\> 
```

# Loop in Reverse

```
Demo.py x ... Python x
D: > Demo.py > ...
1 i = 5
2 while i >= 1:
3     print("i is ", i)
4     i = i-1
5
6
PS D:\> python Demo.py
i is 5
i is 4
i is 3
i is 2
i is 1
PS D:\> 
```

```
i = 5
while i >= 1:
    print("i is ", i)
    i = i-1
```

```
i = 5
while i >= 1:
    print("i is ", i)
    # i=i-2
    i -= 2
```

```
Demo.py x ... Python x
D: > Demo.py > ...
1 i = 5
2 while i >= 1:
3     print("i is ", i)
4     # i=i-2
5     i -= 2
PS D:\> python Demo.py
i is 5
i is 3
i is 1
PS D:\> 
```

# Table

```
loopdemo.py ×
loopdemo.py > ...
1 i = 1
2 no = 5
3 while i<=10:
4     print(no,"*",i,"=",no*i)
5     i=i+1
```

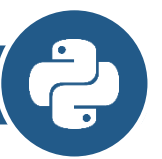
PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
D:\code\python_code>python loopdemo.py
5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
5 * 6 = 30
5 * 7 = 35
5 * 8 = 40
5 * 9 = 45
5 * 10 = 50
```

```
loopdemo.py > ...
1  i = 1
2  number = int(input("Enter No For Table:"))
3  while(i<10):
4      print(number, "*", i, "=", number*i)
5      i = i + 1
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL cmd

```
D:\pythoncode>python loopdemo.py
Enter No For Table:8
8 * 1 = 8
8 * 2 = 16
8 * 3 = 24
8 * 4 = 32
8 * 5 = 40
8 * 6 = 48
8 * 7 = 56
8 * 8 = 64
8 * 9 = 72
```

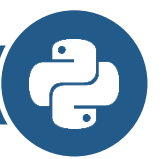


# For Loop



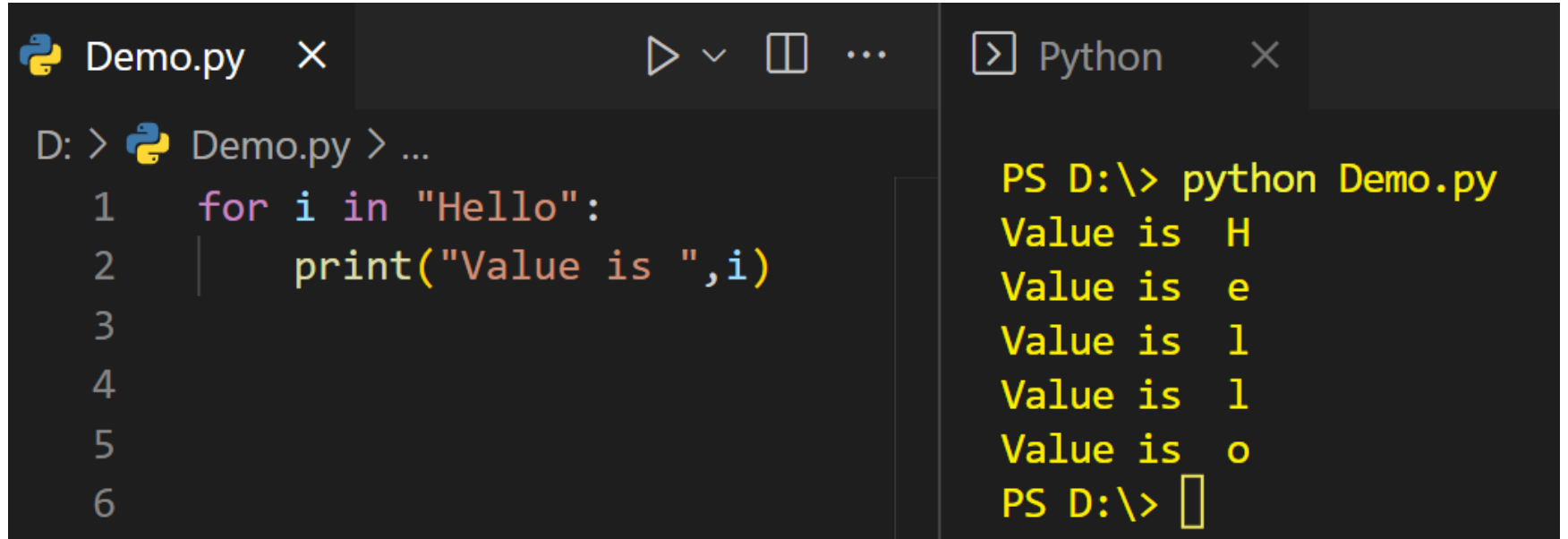
# for loop Statements

- The for loop in Python is used in list, tuple, string.
- **Syntax :-**      for val in sequence:  
                             Statements to be executed



# For loop with String Datatype

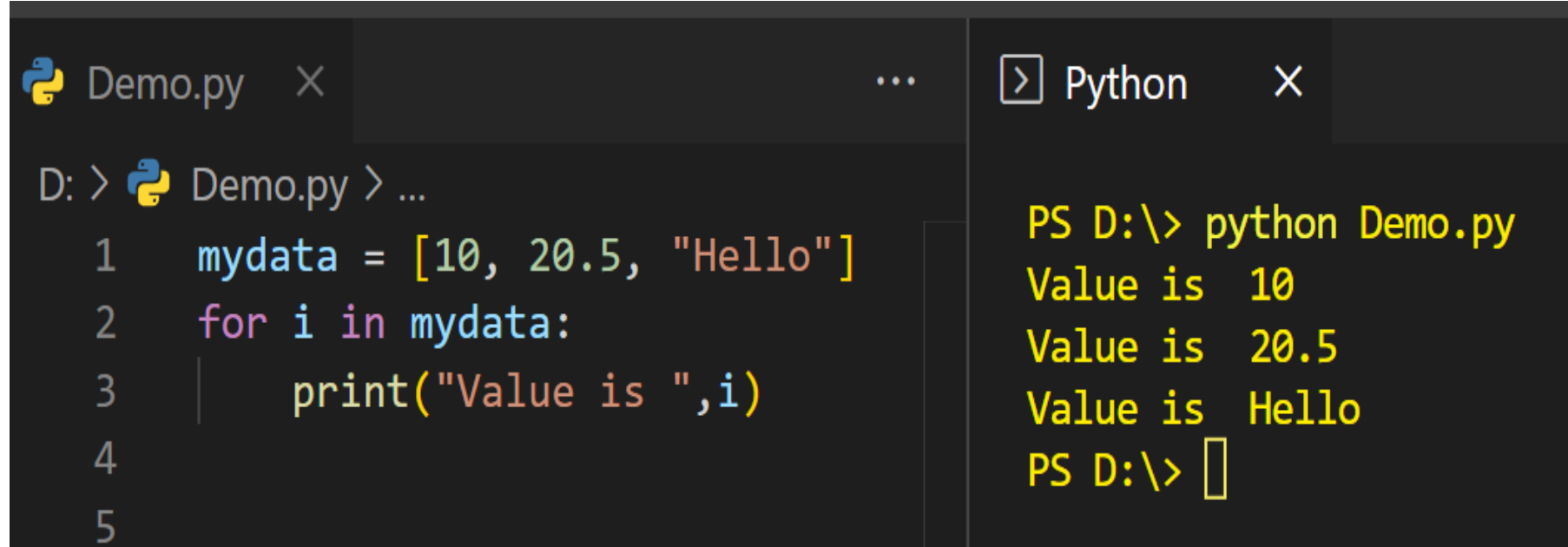
```
for i in "Hello":  
    print("Value is ",i)
```



```
Demo.py × [Run] [Debug] [View] [More] Python ×  
D: > Demo.py > ...  
1   for i in "Hello":  
2   |     print("Value is ",i)  
3  
4  
5  
6  
  
PS D:\> python Demo.py  
Value is  H  
Value is  e  
Value is  l  
Value is  l  
Value is  o  
PS D:\> 
```

# For loop with List Datatype

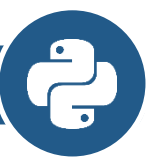
```
mydata = [10, 20.5, "Hello"]  
for i in mydata:  
    print("Value is ",i)
```



```
Demo.py × ... Python ×  
D: > python Demo.py > ...  
1 mydata = [10, 20.5, "Hello"]  
2 for i in mydata:  
3     print("Value is ",i)  
4  
5  
PS D:\> python Demo.py  
Value is 10  
Value is 20.5  
Value is Hello  
PS D:\> 
```

# The range() function

- We can generate a sequence of numbers using range() function.
  - (count).
  - (start,stop).
  - (start,stop,step size).
- 
- Example :- range(10) will generate numbers from 0 to 9 (10 numbers).
- 
- We can also define the start, stop and step size as range(start,stop,step size).  
Step size defaults to 1 if not provided.

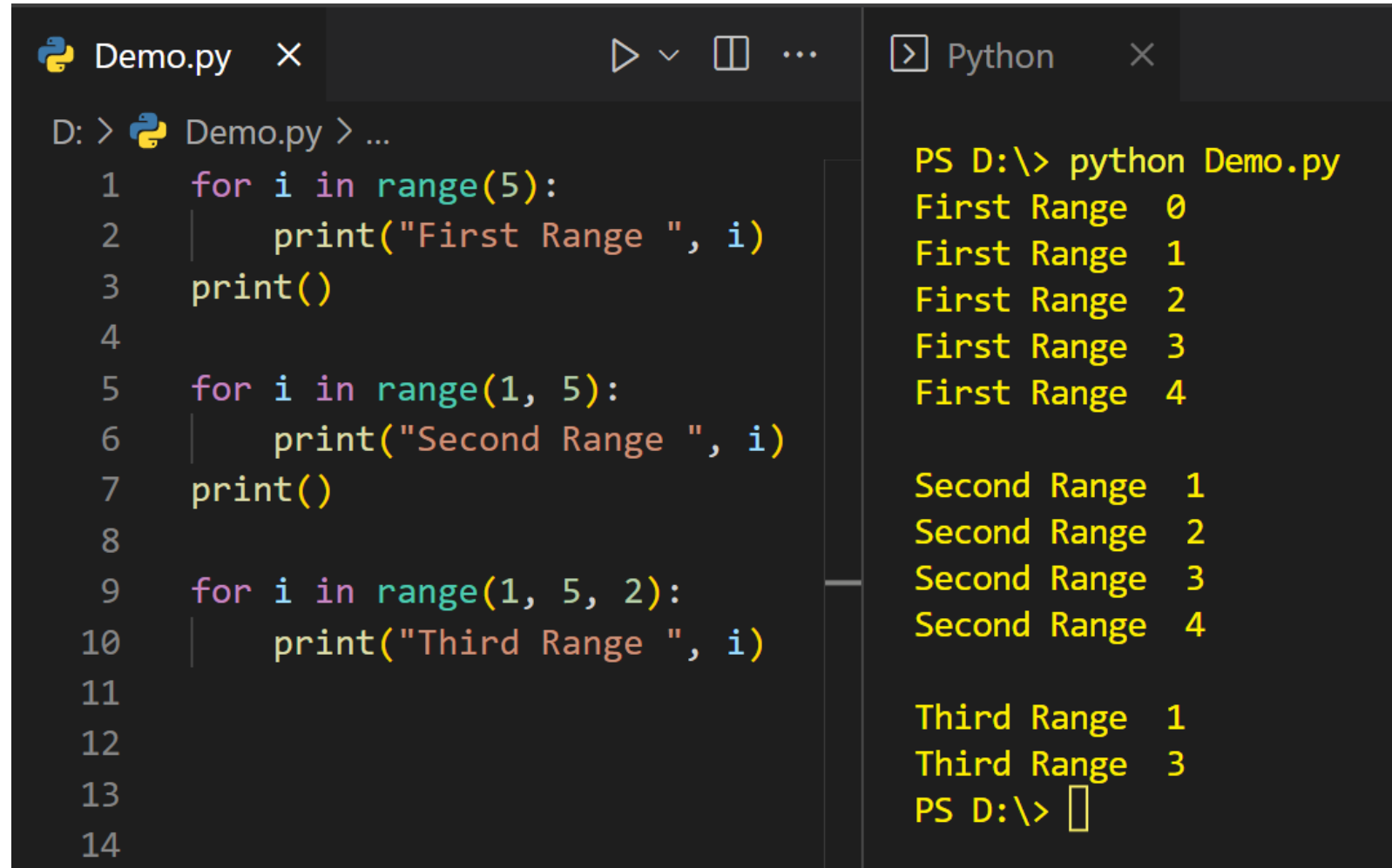


# Range

 loopsdemo.py X  
 loopsdemo.py > ...  
1 for i in range(5):  
2 print("Value is ",i)  
  
PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL  
  
D:\code\python\_code>python loopsdemo.py  
Value is 0  
Value is 1  
Value is 2  
Value is 3  
Value is 4

# Various Range Example

```
for i in range(5):  
    print("First Range ", i)  
print()  
  
for i in range(1, 5):  
    print("Second Range ", i)  
print()  
  
for i in range(1, 5, 2):  
    print("Third Range ", i)
```



```
Demo.py × [Run] [Debug] [Find] [More] Python ×  
D: > Demo.py > ...  
1 for i in range(5):  
2     print("First Range ", i)  
3     print()  
4  
5 for i in range(1, 5):  
6     print("Second Range ", i)  
7     print()  
8  
9 for i in range(1, 5, 2):  
10    print("Third Range ", i)  
11  
12  
13  
14  
  
PS D:\> python Demo.py  
First Range 0  
First Range 1  
First Range 2  
First Range 3  
First Range 4  
  
Second Range 1  
Second Range 2  
Second Range 3  
Second Range 4  
  
Third Range 1  
Third Range 3  
PS D:\>
```

```
Demo.py X
Demo.py > ...
1 for i in range(0,10,2):
2     print("Value is ",i)
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
D:\code\python_code>python Demo.py
Value is 0
Value is 2
Value is 4
Value is 6
Value is 8
```

```
Demo.py > ...
1 for i in range(0,10,3):
2     print("Value is ",i)
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
D:\code\python_code>python Demo.py
Value is 0
Value is 3
Value is 6
Value is 9
```

# Reverse Output

```
Demo.py x [play] [dropdown] [split] [menu]
D: > python Demo.py
1 for i in reversed(range(0, 5, 1)):
2     print("Value is ", i)
3
4
5
6
7
```

```
Python x
PS D:\> python Demo.py
Value is 4
Value is 3
Value is 2
Value is 1
Value is 0
PS D:\>
```

```
for i in reversed(range(0, 5, 1)):
    print("Value is ", i)
```

```
for i in range(5,0,-1):
    print("Value is ",i)
```

```
Demo.py x [play] [dropdown] [split] [menu]
D: > python Demo.py
1 for i in range(5, 0, -1):
2     print("Value is ", i)
3
4
5
6
```

```
Python x
PS D:\> python Demo.py
Value is 5
Value is 4
Value is 3
Value is 2
Value is 1
PS D:\>
```

# Table Print

```
loopdemo.py X
loopdemo.py > ...
1  for i in range(1,11):
2  print("5*",i,"=",5*i)
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
D:\code\python_code>python loopdemo.py
5* 1 = 5
5* 2 = 10
5* 3 = 15
5* 4 = 20
5* 5 = 25
5* 6 = 30
5* 7 = 35
5* 8 = 40
5* 9 = 45
5* 10 = 50
```

```
loopdemo.py > ...
1  i = 1
2  number = int(input("Enter No For Table:"))
3  sno = int(input("Enter Start No:"))
4  eno = int(input("Enter End No:"))
5  for i in range(sno,eno):
6      print(number,"*",i,"=",number*i)
7
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL cmd +

```
D:\pythoncode>python loopdemo.py
Enter No For Table:5
Enter Start No:11
Enter End No:15
5 * 11 = 55
5 * 12 = 60
5 * 13 = 65
5 * 14 = 70
```

## Example 2

- Print List/Tuple value using range():-
  - An alternative way of iterating through each item is by index.
  - For that we have to use **range() function** along with **len() function**.
  - The len() function provides the total number of elements in the tuple , the range() function give us the actual sequence to iterate over.

# Output



```
Demo.py × ... Python ×
D: > Demo.py > ...
1 mydata = [10, 20.5, "Hello"]
2
3 for i in range(len(mydata)):
4     print("Value is ", mydata[i])
5

PS D:\> python Demo.py
Value is 10
Value is 20.5
Value is Hello
PS D:\> 
```

```
mydata = [10, 20.5, "Hello"]
```

```
for i in range(len(mydata)):
    print("Value is ", mydata[i])
```

# Loop with Else

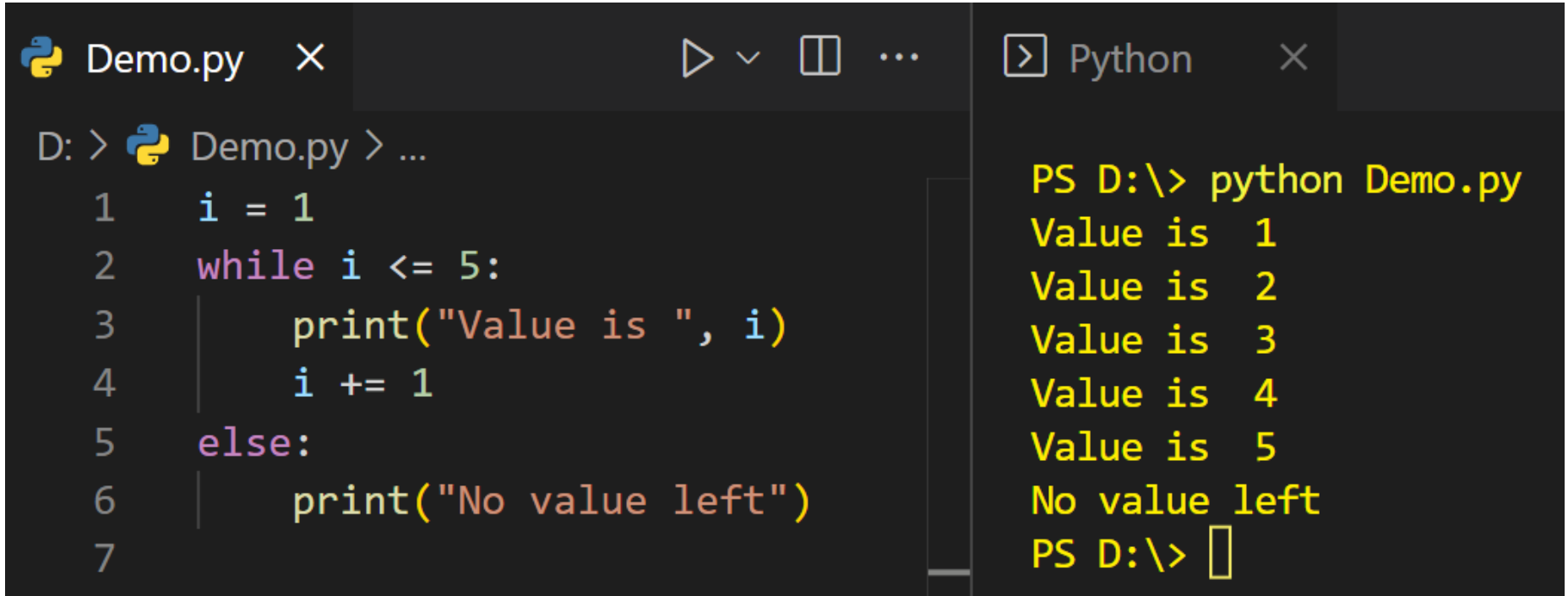


# Loop with else

- In python we can use else block in loop statement.
- **While else: (False part will run else)**
  - If the **else** statement is used with a **while** loop, the **else** statement is executed when the condition becomes false.
- **For Else: (Always else)**
  - If the **else** statement is used with a **for** loop, the **else** statement is executed when the loop has exhausted iterating the list.

# While...else Example

```
i = 1
while i <= 5:
    print("Value is ", i)
    i += 1
else:
    print("No value left")
```



The screenshot shows a Python IDE with two windows. The left window, titled 'Demo.py', contains the following code:

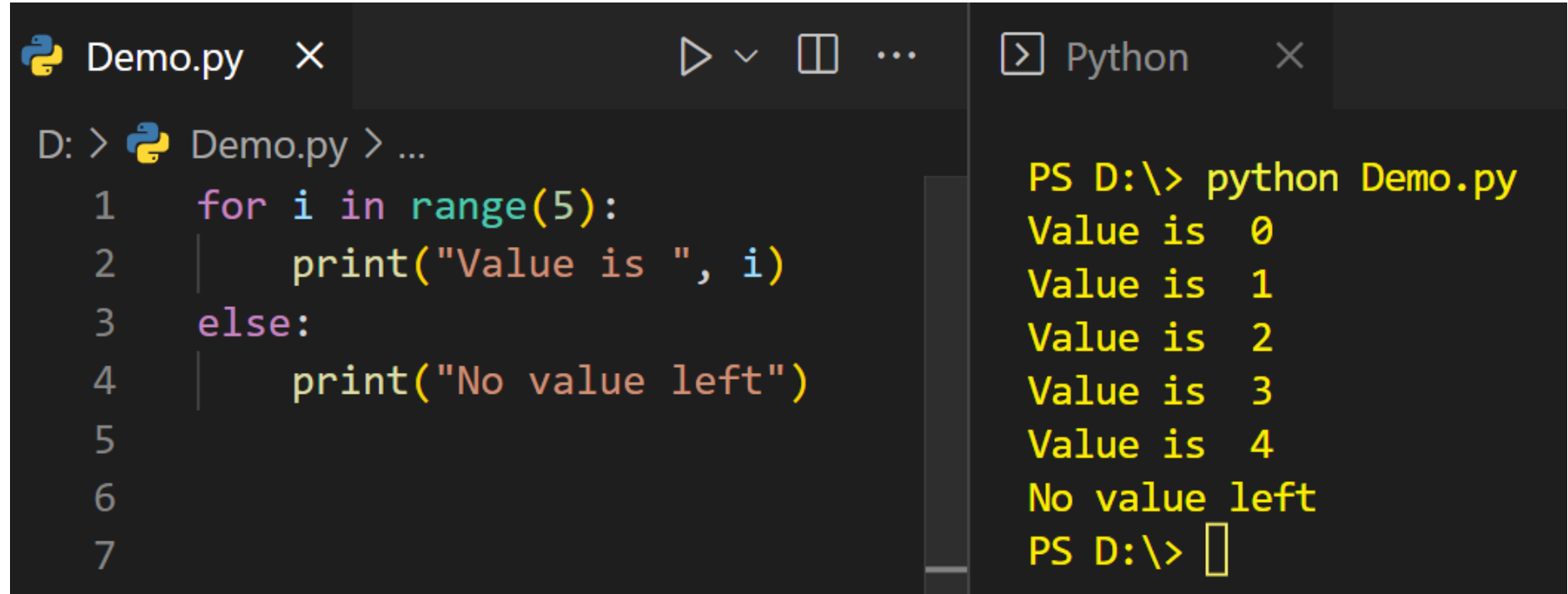
```
D: > python Demo.py > ...
1  i = 1
2  while i <= 5:
3      print("Value is ", i)
4      i += 1
5  else:
6      print("No value left")
7
```

The right window, titled 'Python', shows the output of the script:

```
PS D:\> python Demo.py
Value is 1
Value is 2
Value is 3
Value is 4
Value is 5
No value left
PS D:\>
```

# For...else Example

```
for i in range(5):  
    print("Value is ", i)  
else:  
    print("No value left")
```



```
Demo.py × [Run] [Debug] [Full Screen] [More] Python ×  
D: > Demo.py > ...  
1   for i in range(5):  
2   |       print("Value is ", i)  
3   else:  
4   |       print("No value left")  
5  
6  
7  
  
PS D:\> python Demo.py  
Value is 0  
Value is 1  
Value is 2  
Value is 3  
Value is 4  
No value left  
PS D:\> 
```

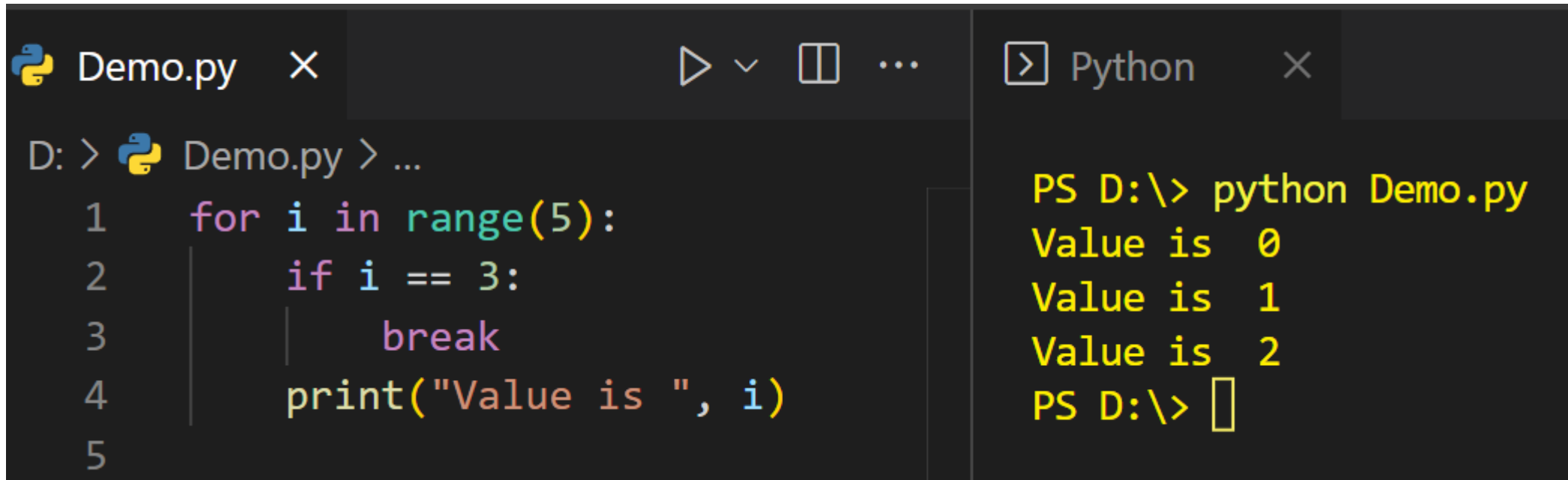
# Break Continue



# "break" and "continue" statements

- **break** is used to exit a for loop or a while loop
- **continue** is used to skip the current block, and return to the "for" or "while" statement.

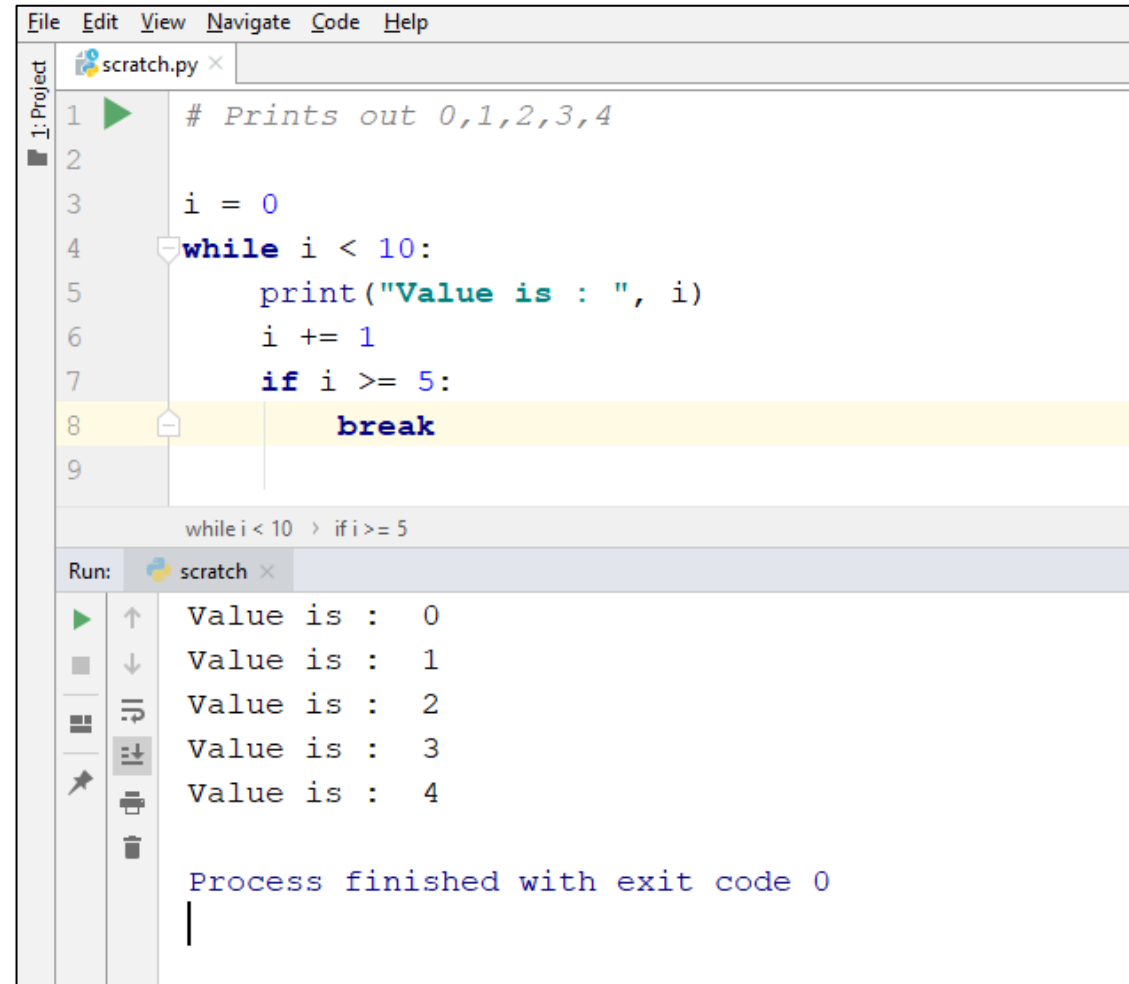
# Example Of Break Statement



```
Demo.py X [Run] [Debug] [View] ... Python X  
D: > Demo.py > ...  
1 for i in range(5):  
2     if i == 3:  
3         break  
4     print("Value is ", i)  
5  
PS D:\> python Demo.py  
Value is 0  
Value is 1  
Value is 2  
PS D:\> 
```

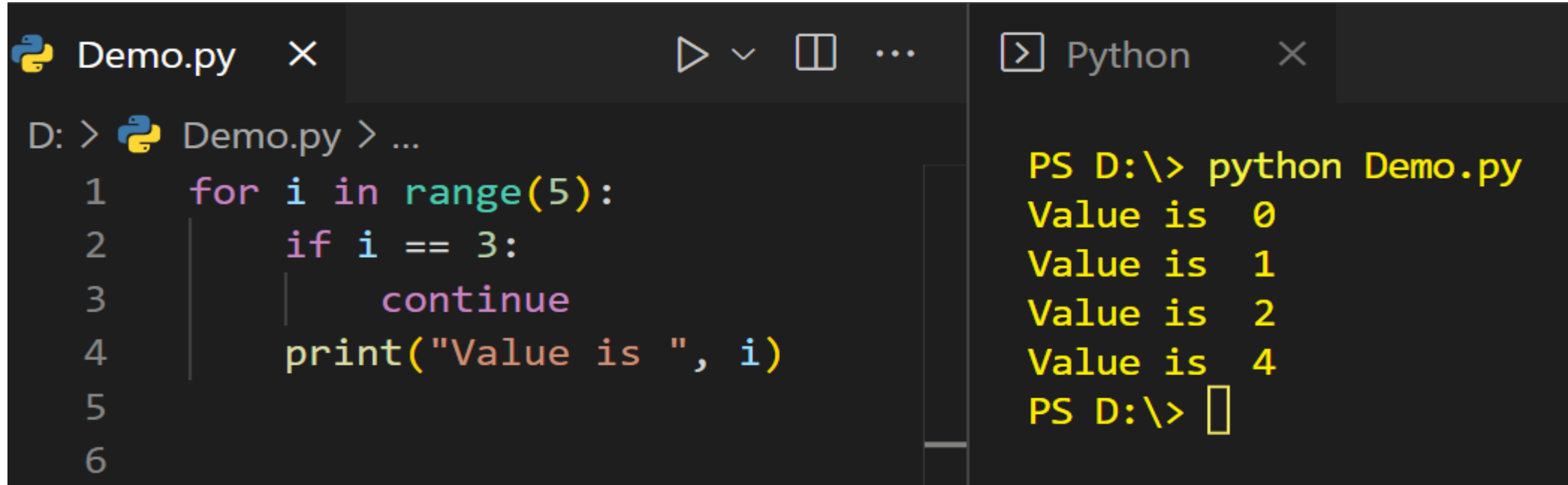
```
for i in range(5):  
    if i == 3:  
        break  
    print("Value is ", i)
```

# Example2



```
File Edit View Navigate Code Help
scratch.py x
1 # Prints out 0,1,2,3,4
2
3 i = 0
4 while i < 10:
5     print("Value is : ", i)
6     i += 1
7     if i >= 5:
8         break
9
while i < 10 > if i >= 5
Run: scratch x
↑ Value is : 0
↓ Value is : 1
↻ Value is : 2
↻ Value is : 3
↻ Value is : 4
Process finished with exit code 0
```

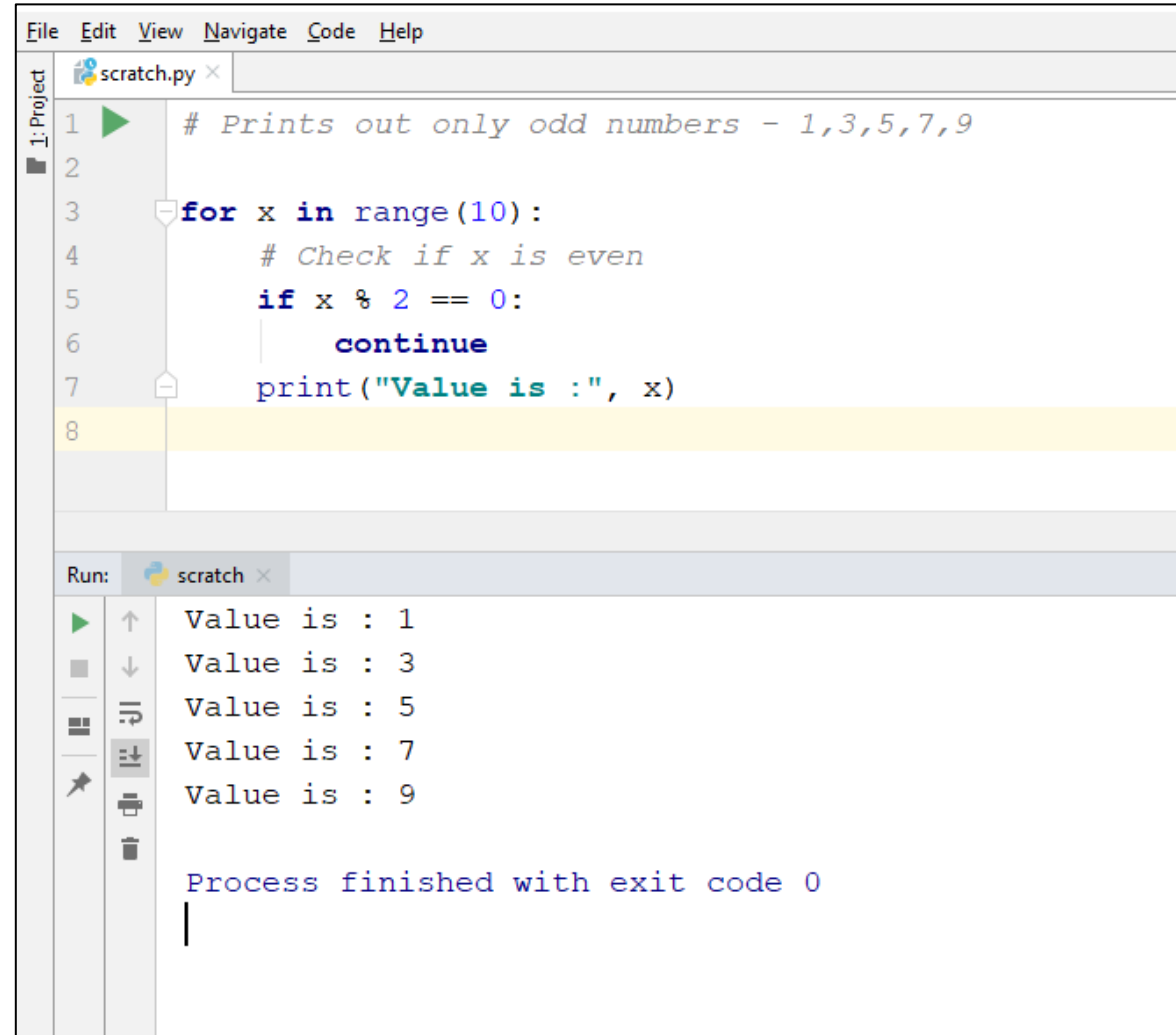
# Example Of Continue Statement



```
Demo.py × [Run] [Debug] [View] [More] Python ×  
D: > Demo.py > ...  
1 for i in range(5):  
2     if i == 3:  
3         continue  
4     print("Value is ", i)  
5  
6  
PS D:\> python Demo.py  
Value is 0  
Value is 1  
Value is 2  
Value is 4  
PS D:\> 
```

```
for i in range(5):  
    if i == 3:  
        continue  
    print("Value is ", i)
```

# Example2



```
File Edit View Navigate Code Help
scratch.py x
1 # Prints out only odd numbers - 1,3,5,7,9
2
3 for x in range(10):
4     # Check if x is even
5     if x % 2 == 0:
6         continue
7     print("Value is :", x)
8
Run: scratch x
Value is : 1
Value is : 3
Value is : 5
Value is : 7
Value is : 9
Process finished with exit code 0
```



# Pass



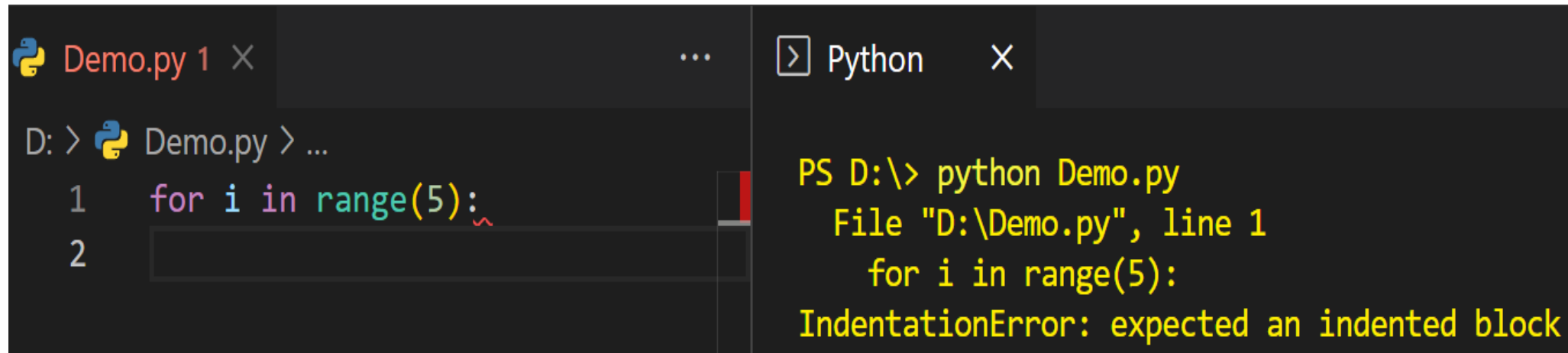
# pass Statement in Python

- pass is a null statement.
- The difference between a comment and pass statement in Python is that, while the interpreter ignores a comment entirely, pass is not ignored.
- It is used as a placeholder.
- **Syntax :-**  
    Pass

# When to use pass statement?

- Suppose we have a loop or a function that is not implemented yet, but we want to implement it in the future.
- They cannot have an empty body. The interpreter would complain.
- So, we use the pass statement to construct a body that does nothing.

# Without Pass Statement(Error)

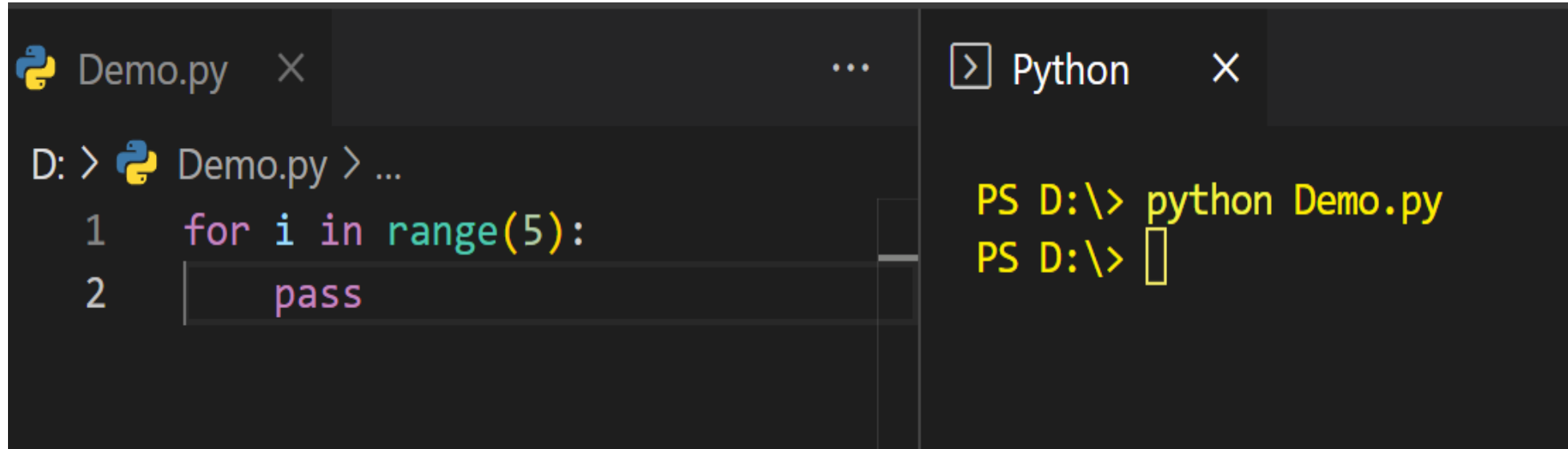


The screenshot shows a code editor with a file named `Demo.py` open. The code in the editor is:

```
1 for i in range(5):  
2
```

The terminal output shows the command `python Demo.py` being executed, resulting in an `IndentationError: expected an indented block`.

# With Pass Statement



```
Demo.py x ... Python x
D: > python Demo.py > ...
1   for i in range(5):
2       pass

PS D:\> python Demo.py
PS D:\> 
```

- With pass statement there will be no error while executing program.
- In output nothing will be print as we have not use any print statement.

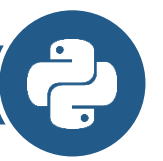
- Take a Number from User and Print Table from 1 to 10
- Take a Number from User and Print Table of given number but table start number and end number will enter by user.
- Take 2 Number Start Number and End Number and Display Only Even Number from given numbers.

```
1 i = 0
2 no = 5
3 while i < 10:
4     i=i+1
5     print(5,"*",i,"=",no*i)
6
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

D:\code\python\_code>python loopdemo.py

```
5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
5 * 6 = 30
5 * 7 = 35
5 * 8 = 40
5 * 9 = 45
5 * 10 = 50
```



```
# Running outer loop from 2 to 3
for i in range(1, 4):

    # Printing inside the outer loop
    # Running inner loop from 1 to 10
    for j in range(1, 11):

        # Printing inside the inner loop
        print(i, "*", j, "=", i*j)
    # Printing inside the outer loop
    print()
```

 loopsdemo.py > ...

```
1  for i in range(1, 6):  
2      for j in range(1, i + 1):  
3          print(j, end="")  
4      print()
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

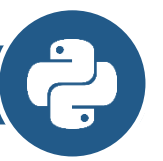
PORTS

TERMINAL

```
D:\code\python_code>python loopsdemo.py
```

```
1  
12  
123  
1234  
12345
```

```
for i in range(1, 6):  
    for j in range(1, i + 1):  
        print(j, end="")  
    print()
```



# Let's Connect



## Akash Padhiyar

Tech Educator • Consultant Tech  
Mentor • Founder

On a Mission 🚀 to Bridge the  
Gap Between Campus & Corporate

**Akashsir.com**

### SOCIAL MEDIA

#DailyUpdates



**@akashsirdotcom**

#FreeCode



**@akashpadhiyar**

#TechUpdates



**@akashpadhiyar**

#FreeTutorials



**@akashtecholabs**

#HelpDesk



**+91 99786 21654**

### WEBSITES

> **www.akashtecholabs.com**

> **www.akashsir.com**



**akash@akashtecholabs.com**

📍 202, Arya Arcade, Mithakhali Six Road, Navrangpura, Ahmedabad, Gujarat 380009